INTERVIEW PREPARATION SERIES

Java Exception Handling

Scenario-Based Questions — 20 Real-World Scenarios

This document presents **20 real-world scenarios** spanning connection leaks, custom exception hierarchies, retry patterns, batch error handling, CompletableFuture failures, stream/lambda checked exceptions, transaction rollback, microservice error frameworks, and production debugging. Each scenario simulates a real-world Java engineering situation.

Resource Management & Cleanup

SCENARIO 01

A production service leaks database connections. After 8 hours, the connection pool is exhausted and the service returns 503 errors. The code uses try-catch but connections are not closed when exceptions occur in the middle of a method. Diagnose and fix.

EXPECTED DEPTH

Bug:

```
// LEAKS: connection not closed if query throws
public List<User> getUsers() {
    Connection conn = dataSource.getConnection();
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery();  // throws here!
    // conn.close() never reached
    List<User> users = mapResults(rs);
    conn.close();
    return users;
}
```

Fix — try-with-resources:

```
public List<User> getUsers() throws SQLException {
    try (Connection conn = dataSource.getConnection();
         PreparedStatement stmt = conn.prepareStatement(sql);
         ResultSet rs = stmt.executeQuery()) {
```

```
        return mapResults(rs);
    }  // ALL three closed automatically, even on exception
}
```

The fix guarantees closure of **all three resources** in reverse order, even if `executeQuery()` or `mapResults()` throws.

SCENARIO 02

A microservice catches all exceptions with `catch (Exception e)` and returns a generic 500 error. The frontend team complains they cannot tell whether it is a validation error (400), authentication error (401), or server error (500). Redesign the exception handling.

EXPECTED DEPTH

Current (broken):

```
try { processOrder(request); }
catch (Exception e) { return Response.status(500).build(); }  // everything
is 500
```

Fixed — Custom exception hierarchy:

```
// Exception hierarchy with HTTP status
public class AppException extends RuntimeException {
    private final int status;
    public AppException(int status, String msg) { super(msg);
this.status=status; }
}
public class ValidationException extends AppException {
    public ValidationException(String msg) { super(400, msg); }
}
public class AuthException extends AppException {
    public AuthException(String msg) { super(401, msg); }
}

// Global handler
@ExceptionHandler(AppException.class)
public ResponseEntity<ErrorBody> handle(AppException e) {
    return ResponseEntity.status(e.getStatus())
        .body(new ErrorBody(e.getMessage()));
}
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorBody> handleUnexpected(Exception e) {
    logger.error("Unhandled", e);
```

```
    return ResponseEntity.status(500).body(new ErrorBody("Internal
error"));
}
```

Production Debugging

SCENARIO 03

A batch job processes 100,000 records. It fails after record 73,412 with a NullPointerException, but the exception message does not indicate WHICH record caused the failure. The developer reruns the entire batch to reproduce. Design better error handling.

EXPECTED DEPTH

```
// WRONG: no context about which record failed
for (Record r : records) { processRecord(r); }

// RIGHT: wrap with context, continue processing, collect errors
List<ProcessingError> errors = new ArrayList<>();
for (int i = 0; i < records.size(); i++) {
    try {
        processRecord(records.get(i));
    } catch (Exception e) {
        errors.add(new ProcessingError(i, records.get(i).getId(), e));
        logger.error("Failed record {} (index {}): {}",
            records.get(i).getId(), i, e.getMessage());
        // Continue processing remaining records!
    }
}
logger.info("Processed {}, failed {}", records.size(), errors.size());
```

Three improvements: (1) include record ID and index in the error, (2) continue processing remaining records, (3) collect all errors for a summary report.

SCENARIO 04

An API endpoint intermittently throws `java.net.SocketTimeoutException` when calling a downstream service.

The current code catches it and returns a 500 error. Implement retry with exponential backoff.

EXPECTED DEPTH

```
public <T> T retryWithBackoff(Callable<T> operation, int maxRetries) {
    int attempt = 0;
    while (true) {
        try {
            return operation.call();
        } catch (SocketTimeoutException | ConnectException e) {
            attempt++;
            if (attempt >= maxRetries) {
                throw new ServiceUnavailableException(
                    "Failed after " + maxRetries + " retries", e);
            }
            long delay = (long) Math.pow(2, attempt) * 1000; // 2s, 4s, 8s
            logger.warn("Attempt {} failed, retrying in {}ms", attempt,
delay);
            Thread.sleep(delay);
        } catch (Exception e) {
            throw new RuntimeException("Non-retryable error", e);
        }
    }
}

// Usage
User user = retryWithBackoff(() -> userService.fetchUser(id), 3);
```

Key: only retry **transient failures** (timeout, connection refused). Do not retry permanent failures (400 Bad Request, 404 Not Found).

Custom Exception Design

SCENARIO 05

Your e-commerce platform has 12 microservices. Each throws different unstructured exceptions. The API gateway cannot produce consistent error responses. Design a unified exception framework.

EXPECTED DEPTH

```
// Shared library: unified exception hierarchy
public abstract class PlatformException extends RuntimeException {
```

```
    private final String errorCode;  // e.g., "ORDER-001"
    private final int httpStatus;
    private final Map<String, Object> metadata;

    protected PlatformException(String code, int status, String msg,
                                Map<String, Object> meta, Throwable cause)
{
        super(msg, cause);
        this.errorCode = code; this.httpStatus = status;
        this.metadata = meta != null ? meta : Map.of();
    }
}

// Per-service exceptions extend the base
public class OrderNotFoundException extends PlatformException {
    public OrderNotFoundException(String orderId) {
        super("ORDER-001", 404, "Order not found: " + orderId,
              Map.of("orderId", orderId), null);
    }
}

// API Gateway: consistent error response
{ "error": "ORDER-001", "message": "Order not found: ORD-123",
  "metadata": { "orderId": "ORD-123" } }
```

The error code (ORDER-001) enables automated monitoring, the metadata enables debugging, and the HTTP status enables correct client behavior. All 12 services use the same format.

Exception Chaining & Multi-Threading

SCENARIO 06

A developer wraps a SQLException in a RuntimeException but forgets to include the cause. Production logs show `RuntimeException: Error` with no stack trace of the original database error. The DBA cannot diagnose the issue. Fix the exception chaining.

EXPECTED DEPTH

```
// WRONG: cause lost
catch (SQLException e) {
    throw new RuntimeException("Database error");  // original cause GONE
```

```
    // Log shows: RuntimeException: Database error
    // No information about WHICH SQL failed or WHY
}

// CORRECT: chain the cause
catch (SQLException e) {
    throw new RuntimeException("Database error querying users table", e);
    // Log shows:
    // RuntimeException: Database error querying users table
    //   Caused by: SQLException: Connection refused to host db-prod:5432
    //     at org.postgresql.Driver.connect(...)
}
```

The chained stack trace gives the DBA everything they need: the application context (which operation failed) AND the database context (which connection/query failed).

SCENARIO 07

Your application uses `CompletableFuture` for parallel API calls. When one call fails, the entire pipeline fails silently — no error is logged. The result is null, causing NullPointerException downstream. Fix the async error handling.

EXPECTED DEPTH

```
// BROKEN: exception silently swallowed
CompletableFuture<User> user = CompletableFuture.supplyAsync(() ->
fetchUser(id));
CompletableFuture<Orders> orders = CompletableFuture.supplyAsync(() ->
fetchOrders(id));
// If fetchUser throws, user.join() throws CompletionException
// But the code just returns null!

// FIXED: proper async error handling
CompletableFuture<Dashboard> dashboard = CompletableFuture
    .supplyAsync(() -> fetchUser(id))
    .thenCombine(
        CompletableFuture.supplyAsync(() -> fetchOrders(id)),
        (user, orders) -> buildDashboard(user, orders)
    )
    .exceptionally(ex -> {
        logger.error("Dashboard build failed for user " + id, ex);
        return Dashboard.empty();  // fallback
    });
```

```
// Or: fail fast
try { dashboard.join(); }
catch (CompletionException e) {
    throw new ServiceException("Dashboard failed", e.getCause());
}
```

Stream & Lambda Exceptions

SCENARIO 08

A data processing pipeline uses Java streams to read and transform files. The `Files.readString()` call throws `IOException`, but streams do not support checked exceptions. The code wraps every lambda in try-catch, making it unreadable. Design a cleaner solution.

EXPECTED DEPTH

```
// UGLY: try-catch in every lambda
paths.stream().map(p -> {
    try { return Files.readString(p); }
    catch (IOException e) { throw new UncheckedIOException(e); }
}).map(content -> {
    try { return objectMapper.readValue(content, Record.class); }
    catch (JsonProcessingException e) { throw new RuntimeException(e); }
})

// CLEAN: utility wrapper method
@FunctionalInterface
interface CheckedFunction<T, R> { R apply(T t) throws Exception; }

static <T, R> Function<T, R> unchecked(CheckedFunction<T, R> f) {
    return t -> { try { return f.apply(t); }
                  catch (Exception e) { throw new RuntimeException(e); } };
}

// Now the pipeline is readable:
paths.stream()
    .map(unchecked(Files::readString))
    .map(unchecked(c -> objectMapper.readValue(c, Record.class)))
    .collect(toList());
```

Error Recovery & Resilience

SCENARIO 09

A payment service processes charges via a third-party API. Occasionally the API returns a timeout, but the charge was actually processed. Retrying creates a double charge. Design idempotent error handling.

EXPECTED DEPTH

```
public ChargeResult processCharge(String idempotencyKey, BigDecimal amount)
{
    // Check if already processed (idempotency)
    Optional<ChargeResult> existing = chargeRepo.findByKey(idempotencyKey);
    if (existing.isPresent()) return existing.get();  // return cached
result

    try {
        PaymentResponse resp = paymentApi.charge(amount, idempotencyKey);
        ChargeResult result =
ChargeResult.success(resp.getTransactionId());
        chargeRepo.save(idempotencyKey, result);
        return result;
    } catch (SocketTimeoutException e) {
        // Timeout: charge MAY have been processed
        // DO NOT retry blindly!
        return ChargeResult.uncertain("Timeout: verify with payment
provider");
    } catch (PaymentDeclinedException e) {
        ChargeResult result = ChargeResult.declined(e.getReason());
        chargeRepo.save(idempotencyKey, result);
        return result;
    }
}
```

The idempotency key ensures the same charge request always returns the same result, even if retried. The timeout case returns “uncertain” instead of retrying — preventing double charges.

SCENARIO 10

A scheduled batch job runs nightly. If it fails, the operations team does not know until users complain the next morning. Implement proper error notification and monitoring.

EXPECTED DEPTH

```
@Scheduled(cron = "0 0 2 * * *")  // 2 AM daily
public void nightlyBatch() {
    BatchResult result = new BatchResult();
    try {
        result = processBatch();
        if (result.hasErrors()) {
            alertService.warn("Batch completed with " + result.errorCount()
+ " errors");
        }
        metrics.recordBatchSuccess(result.processedCount());
    } catch (Exception e) {
        logger.error("BATCH FAILED", e);
        alertService.critical("Nightly batch FAILED: " + e.getMessage());
        metrics.recordBatchFailure();
        // Optionally: retry once
        // throw e;  // let scheduler handle retry
    } finally {
        auditLog.record("nightly-batch", result);
    }
}
```

Three notification channels: logging (for investigation), alerting (PagerDuty/Slack for immediate response), and metrics (Grafana dashboard for trends).

Advanced Exception Patterns

SCENARIO 11

Your REST API must validate 15 fields in a request body. The current code throws an exception on the first invalid field, forcing the client to fix one field at a time. Design a validation that collects all errors and returns them together.

EXPECTED DEPTH

```
public List<ValidationError> validate(CreateUserRequest req) {
    List<ValidationError> errors = new ArrayList<>();
    if (req.getEmail() == null || !req.getEmail().contains("@"))
        errors.add(new ValidationError("email", "Invalid email"));
    if (req.getAge() != null && req.getAge() < 0)
        errors.add(new ValidationError("age", "Must be non-negative"));
    if (req.getName() == null || req.getName().isBlank())
```

```
            errors.add(new ValidationError("name", "Required"));
        // ... all 15 fields
        if (!errors.isEmpty())
            throw new ValidationException(errors);  // all errors at once
        return errors;
    }

    // Response: {"errors": [{"field":"email","msg":"Invalid"},
    {"field":"name","msg":"Required"}]}
```

This is the **notification pattern**: collect all validation errors and throw once, vs the **fail-fast pattern** which throws on the first error.

SCENARIO 12

A critical financial calculation must NEVER silently produce wrong results. If anything goes wrong, it must fail loudly with full context. Design defensive exception handling for a tax calculation service.

EXPECTED DEPTH

```
public TaxResult calculateTax(TaxRequest request) {
    Objects.requireNonNull(request, "TaxRequest cannot be null");
    Objects.requireNonNull(request.getIncome(), "Income cannot be null");
    if (request.getIncome().compareTo(BigDecimal.ZERO) < 0)
        throw new IllegalArgumentException("Income cannot be negative: " +
request.getIncome());

    try {
        BigDecimal tax = taxEngine.compute(request);
        // Post-condition check
        if (tax.compareTo(request.getIncome()) > 0)
            throw new IllegalStateException(
                "Computed tax (" + tax + ") exceeds income (" +
request.getIncome() + ")");
        return new TaxResult(tax, request);
    } catch (ArithmeticException e) {
        throw new TaxCalculationException(
            "Arithmetic error for income=" + request.getIncome(), e);
    }
}
```

Three layers of defense: precondition checks (null/range), computation with catch, and postcondition assertions. For financial code, **failing loudly is safer than returning a wrong number silently**.

SCENARIO 13

Your application uses a third-party library that throws generic `Exception` for everything. You need to distinguish between network errors (retryable) and data errors (not retryable). Design a wrapper.

EXPECTED DEPTH

```
public <T> T callLibrary(Callable<T> operation) {
    try {
        return operation.call();
    } catch (Exception e) {
        // Classify by examining the exception chain
        if (isNetworkError(e))
            throw new RetryableException("Network error", e);
        if (isDataError(e))
            throw new DataException("Data error", e);
        throw new UnknownLibraryException("Unexpected library error", e);
    }
}

private boolean isNetworkError(Throwable t) {
    while (t != null) {
        if (t instanceof SocketTimeoutException
         || t instanceof ConnectException
         || t.getMessage().contains("connection refused"))
            return true;
        t = t.getCause();
    }
    return false;
}
```

SCENARIO 14

During load testing, your application throws OutOfMemoryError after processing 50,000 requests. The error happens in a specific endpoint. How do you diagnose the memory leak?

EXPECTED DEPTH

Step 1 — Capture heap dump:

```
# JVM flag: auto-dump on OOM
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/heapdump.hprof -
jar app.jar

# Manual dump while running
jmap -dump:format=b,file=/tmp/heap.hprof <PID>
```

Step 2 — Analyze with Eclipse MAT or VisualVM:

- Find the largest retained objects
- Look for collections (List, Map) growing unboundedly
- Check for unclosed InputStreams, connections, or event listeners

Common causes:

- Unclosed resources (streams, connections) in exception paths
- Unbounded caches (HashMap without eviction)
- Event listener references preventing garbage collection
- ThreadLocal variables not cleaned up in thread pools

SCENARIO 15

A Spring Boot application logs exceptions at every layer (controller, service, repository), resulting in the same exception being logged 3 times. The logs are unusable. Fix the logging strategy.

EXPECTED DEPTH

Anti-pattern: log at every layer

```
// Repository
catch (SQLException e) { logger.error("DB error", e); throw new
DataException(e); }
// Service
catch (DataException e) { logger.error("Service error", e); throw new
ServiceException(e); }
// Controller
catch (ServiceException e) { logger.error("Request error", e); return
error(); }
```

Fix: log at the boundary only

```
// Repository: translate, don't log
catch (SQLException e) { throw new DataException("Query failed", e); }
// Service: translate, don't log
```

```
catch (DataException e) { throw new ServiceException("User lookup failed",
e); }
// Controller advice: log ONCE at the boundary
@ExceptionHandler(ServiceException.class)
public ResponseEntity handle(ServiceException e) {
    logger.error("Request failed", e);  // logged ONCE with full chain
    return ResponseEntity.status(500).body(errorBody);
}
```

Rule: translate exceptions at each layer (add context), but log only at the **outermost boundary** (controller advice, error handler).

Exception Testing & Edge Cases

SCENARIO 16

A test passes in the IDE but fails in CI because an exception is thrown during static initialization of a test dependency. The CI shows `ExceptionInInitializerError` with a confusing stack trace. Diagnose and fix.

EXPECTED DEPTH

Root cause: a static initializer loads a config file: `static { props = loadFile("/config.properties"); }`. The file exists in the IDE's classpath but not in the CI container.

Fix:

```
// WRONG: file path in static initializer
static {
    props = loadFile("/config.properties"); // ExceptionInInitializerError
in CI
}

// CORRECT: classpath resource loading with fallback
static {
    try (InputStream is =
MyClass.class.getResourceAsStream("/config.properties")) {
        if (is != null) { props.load(is); }
        else { props = getDefaultProperties(); }
    } catch (IOException e) {
        throw new ExceptionInInitializerError(e);
    }
}
```

Use classpath resources (`getResourceAsStream`) instead of filesystem paths. Always handle the null case (resource not found) with a sensible default.

SCENARIO 17

A method is supposed to throw `IllegalArgumentException` for negative input, but it actually throws `NullPointerException` when the input is null (because the null check comes after the range check). Write proper defensive code and tests.

EXPECTED DEPTH

```
// BUG: NPE before range check
public void setAge(Integer age) {
    if (age < 0) throw new IllegalArgumentException("Negative");
    // NPE if age is null because unboxing null → NPE
    this.age = age;
}

// FIXED: check null first
public void setAge(Integer age) {
    Objects.requireNonNull(age, "age must not be null");
    if (age < 0) throw new IllegalArgumentException("age must be non-
negative: " + age);
    this.age = age;
}

// Tests
@Test void nullAgeShouldThrowNPE() {
    assertThrows(NullPointerException.class, () -> user.setAge(null));
}
@Test void negativeAgeShouldThrowIAE() {
    assertThrows(IllegalArgumentException.class, () -> user.setAge(-5));
}
```

SCENARIO 18

Your application catches `InterruptedException` in a loop but does not restore the interrupt flag. The thread pool's shutdown is delayed because threads ignore interrupts. Fix this.

EXPECTED DEPTH

```
// WRONG: interrupt signal lost
```

```
while (running) {
    try { Thread.sleep(1000); }
    catch (InterruptedException e) {
        logger.warn("Interrupted");
        // Thread.interrupted flag is now FALSE
        // Thread pool cannot shut down this thread!
    }
}

// CORRECT: restore interrupt flag
while (running) {
    try { Thread.sleep(1000); }
    catch (InterruptedException e) {
        logger.warn("Interrupted, shutting down");
        Thread.currentThread().interrupt();  // restore the flag
        running = false;  // exit the loop
    }
}
```

Rule: if you catch InterruptedException and do not rethrow it, **always call Thread.currentThread().interrupt()** to restore the flag.

SCENARIO 19

A complex business transaction must either complete fully or roll back all changes. If any step fails, all previous steps must be undone. Design the exception handling for this saga pattern.

EXPECTED DEPTH

```
public OrderResult createOrder(OrderRequest req) {
    String orderId = null;
    String paymentId = null;
    String shippingId = null;
    try {
        orderId = orderService.create(req);
        paymentId = paymentService.charge(req.getAmount(), orderId);
        shippingId = shippingService.schedule(orderId, req.getAddress());
        return OrderResult.success(orderId, paymentId, shippingId);
    } catch (Exception e) {
        // Compensating transactions (reverse order)
        if (shippingId != null) shippingService.cancel(shippingId);
        if (paymentId != null) paymentService.refund(paymentId);
        if (orderId != null) orderService.cancel(orderId);
        throw new OrderFailedException("Order creation failed", e);
```

```
    }
}
```

This is the **saga pattern with compensating transactions**. Each step has a corresponding undo action. If step 3 fails, steps 2 and 1 are reversed.

SCENARIO 20

Your team is migrating from checked exceptions (IOException, SQLException) to unchecked exceptions throughout the codebase. A junior developer asks: “Won’t we miss errors if the compiler doesn’t force us to catch them?” How do you answer?

EXPECTED DEPTH

The argument for unchecked:

- Checked exceptions force **every intermediate method** to either catch or declare. In a 10-layer architecture, throws clauses propagate through 8 layers that cannot meaningfully handle the exception
- Most catch blocks either rethrow, wrap, or log — they do not actually **recover**
- Checked exceptions interact poorly with lambdas, streams, and functional programming
- Spring, Hibernate, and modern Java frameworks already use unchecked exceptions

How to stay safe without checked exceptions:

- **Global exception handlers** (@ControllerAdvice) catch everything at the boundary
- **Custom exception hierarchy** with error codes enables consistent error handling
- **Unit tests** (assertThrows) verify exception behavior
- **Code reviews** catch missing error handling
- **Monitoring and alerting** catch unexpected exceptions in production

The compiler check is one safety net. It is replaced by multiple stronger safety nets: tests, handlers, monitoring.
